

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

Facultad de Ingeniería

Escuela Profesional de Ingeniería en Informática y Sistemas



PRÁCTICA N° 03:

“CREACIÓN DE PROCESOS BAJO LINUX”

Alumno	:	Mauricio José Flores Gómez
Código	:	2018-119061
Docente	:	Dr. Hugo Manuel Barraza Vizcarra
Curso	:	Sistemas Operativos
Ciclo	:	6to Ciclo
Año de estudios	:	3er año

Tacna - Perú

2026

I. OBJETIVOS

Al finalizar la práctica, el estudiante será capaz de :

- Identificar procesos y reconocer sus identificadores PID y PPID en un sistema Linux.
- Comprender la relación existente entre un proceso padre y sus procesos hijos.
- Crear nuevos procesos mediante la llamada al sistema `fork()`.
- Interpretar correctamente el valor retornado por `fork()` para diferenciar la ejecución del proceso padre y del proceso hijo.
- Reemplazar la imagen de un proceso mediante la familia de funciones `exec()`.
- Sincronizar la ejecución de procesos padre e hijo mediante `wait()` o `waitpid()`.
- Analizar la creación, ejecución y finalización de procesos mediante pequeños programas desarrollados en C/C++.

II. FUNDAMENTO TEÓRICO

Procesos de Linux

Un proceso es una instancia de un programa en ejecución. Cada proceso posee su propio contexto de ejecución, que incluye, entre otros elementos.

- Código del programa
- Datos
- Pila
- Registros del procesador
- Archivos abiertos
- Estado del proceso
- Identificador del proceso

Linux asigna a cada proceso un identificador denominado PID (Process Identifier).

Asimismo, cada proceso puede estar relacionado con el proceso que lo creó. El identificador de dicho proceso se denomina PPID (Parent Process Identifier).

Los procesos forman, por tanto, una estructura jerárquica de relaciones padre-hijo.

Llamadas al sistema

Las llamadas al sistema constituyen el mecanismo mediante el cual un programa solicita servicios al núcleo del sistema operativo.

Entre las operaciones que requieren la intervención del sistema operativo se encuentran:

- Creación y finalización de procesos
- Acceso a archivos
- Operaciones de entrada/salida
- Comunicación entre procesos
- Administración de memoria
- Acceso a dispositivos

En esta práctica se utilizarán principalmente las siguientes funciones POSIX:

Función	Propósito
<code>getpid()</code>	Obtener el PID del proceso actual
<code>getppid()</code>	Obtener el PID del proceso padre
<code>fork()</code>	Crear un nuevo proceso
<code>exec()</code>	Reemplazar el programa que ejecuta un proceso
<code>wait()</code>	Esperar la terminación de un proceso hijo
<code>waitpid()</code>	Esperar la terminación de un hijo específico

Conceptos básicos de multitarea

La forma básica de la multitarea es usar interrupciones, más conocida como multitarea en serie; un programa se detiene temporalmente mientras que a otro se le permite ser ejecutado.

Esta característica trabaja en base a "time slices" o "fracciones de tiempo". A cada proceso se le asigna una pequeña porción de tiempo para ejecutarse.

Unix es un sistema multiproceso por tiempo compartido. Desde el punto de vista del usuario en un momento dado hay múltiples programas en ejecución, cada uno de ellos avanzando en su tarea. Sin embargo en una máquina con un solo procesador (monoprocesador) hay en cada instante solamente un proceso ejecutando. Es el sistema operativo el que va rotando el uso del procesador a intervalos breves (decenas de milisegundos) entre los procesos definidos en el sistema de forma que se crea la ilusión que todos avanzan simultáneamente.

El administrador del sistema dispone de herramientas para supervisar el estado de los procesos y eventualmente tomar acciones para suspender o detener la ejecución de un proceso o simplemente modificar su comportamiento.

Los cambios entre diferentes procesos ocurren tan rápido que nosotros no nos percatamos de ello. Este cambio es posible debido a que Linux asocia a un proceso un número de identificación; pero no solo identifica al número, también identifica a las variables asociadas al proceso. Estas variables quedan en el "limbo" cuando el proceso es suspendido, y las activa de nuevo cuando el procesador reanuda la ejecución del proceso.

Compilación de un programa C en Linux

En esta práctica trabajaremos con las llamadas al sistema básicas `fork()`, `exec()`, `getpid()`, `getppid()` y la implementación en pequeños programas escritos en C++, editados con el Editor de textos (`gedit`) de Ubuntu y compilándolos desde la terminal. Si no tienes los compiladores instalados (Verifique la instalación mediante: `g++ --version`), ejecuta previamente:

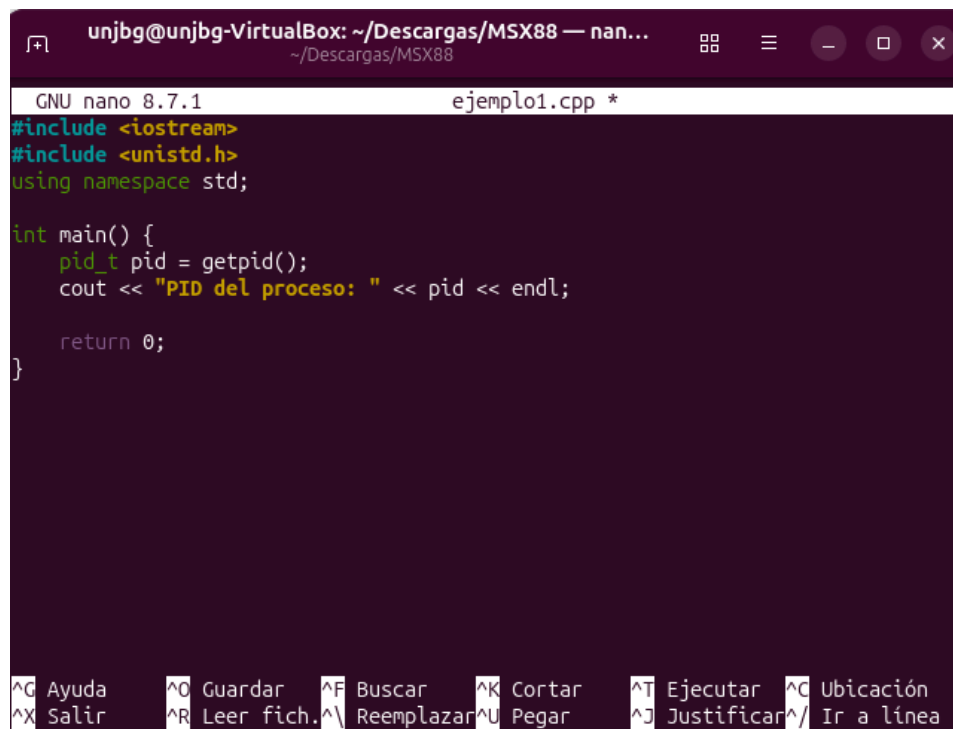
- `sudo apt-get update`
- `sudo apt-get install g++ build-essential`

III. PROCEDIMIENTO

Parte 1: Identificación de un proceso

El sistema operativo Linux asigna a cada proceso un número identificador único y exclusivo que pertenece al proceso en toda su existencia en la memoria. Para obtener el número de identificación de un proceso haremos uso de la función `getpid()`.

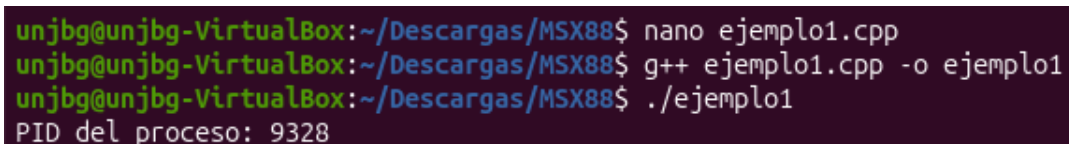
Creamos el archivo `ejemplo1.cpp` mediante el editor Nano e ingresamos el código en C++.



```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nan...  
~/Descargas/MSX88  
GNU nano 8.7.1 ejemplo1.cpp *  
#include <iostream>  
#include <unistd.h>  
using namespace std;  
  
int main() {  
    pid_t pid = getpid();  
    cout << "PID del proceso: " << pid << endl;  
    return 0;  
}
```

^G Ayuda ^O Guardar ^F Buscar ^K Cortar ^T Ejecutar ^C Ubicación
^X Salir ^R Leer fich. ^\ Reemplazar ^U Pegar ^J Justificar ^_ Ir a línea

Compilamos el programa mediante `g++` y lo ejecutamos.



```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ nano ejemplo1.cpp  
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ g++ ejemplo1.cpp -o ejemplo1  
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ ./ejemplo1  
PID del proceso: 9328
```

Ejecutamos el comando `ps -e`.

```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88
8373 ?      00:00:00 kworker/2:2-events
8375 ?      00:00:00 cupsd
8378 ?      00:00:00 cups-browsed
8383 ?      00:00:00 kworker/0:1-ata_sff
8498 ?      00:00:00 kworker/u17:0-ext4-rsv-conversion
8614 ?      00:00:00 kworker/2:1-cgroup_release
8660 ?      00:00:00 kworker/0:2-cgroup_free
8663 ?      00:00:00 kworker/u19:0
8667 ?      00:00:13 ptyxis
8683 ?      00:00:00 ptyxis-agent
8719 pts/0   00:00:00 bash
8732 ?      00:00:00 kworker/1:0-cgroup_release
8747 ?      00:00:00 kworker/u20:3-events_power_efficient
8787 ?      00:00:00 Web Content
8821 ?      00:00:00 Web Content
9041 ?      00:00:00 apt.systemd.dai
9046 ?      00:00:00 apt.systemd.dai
9087 ?      00:00:42 unattended-upgr
9166 ?      00:00:02 http
9220 ?      00:00:00 kworker/3:0-cgroup_free
9268 ?      00:00:00 kworker/2:0
9308 ?      00:00:00 kworker/0:0-events
9330 pts/0   00:00:00 ps
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$
```

Actividad 1

Responda:

1. ¿Qué valor del PID obtuvo al ejecutar el programa?

El valor del PID obtenido fue de 9328

2. ¿El PID obtenido aparece al ejecutar posteriormente `ps -e`?

No, el PID obtenido no aparece

3. Explique por qué no puede aparecer

No puede aparecer porque el programa ya terminó su ejecución. AL finalizar el proceso deja de estar activo y por eso el PID ya no aparece en la lista de procesos mostrada con `ps -e`

Comprobación adicional:

Para mantener el proceso activo durante algunos segundos modificaremos el programa agregando `sleep(20)`; antes del `return 0`.

```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nan...
~/Descargas/MSX88
GNU nano 8.7.1 ejemplo1.cpp *
#include <iostream>
#include <unistd.h>
using namespace std;

int main() {
    pid_t pid = getpid();
    cout << "PID del proceso: " << pid << endl;

    sleep(20);
    return 0;
}
```

^G Ayuda ^O Guardar ^F Buscar ^K Cortar ^T Ejecutar ^C Ubicación
^X Salir ^R Leer fich. ^_ Reemplazar ^U Pegar ^J Justificar ^/ Ir a línea

Compilamos nuevamente el programa mediante g++ y lo ejecutamos.

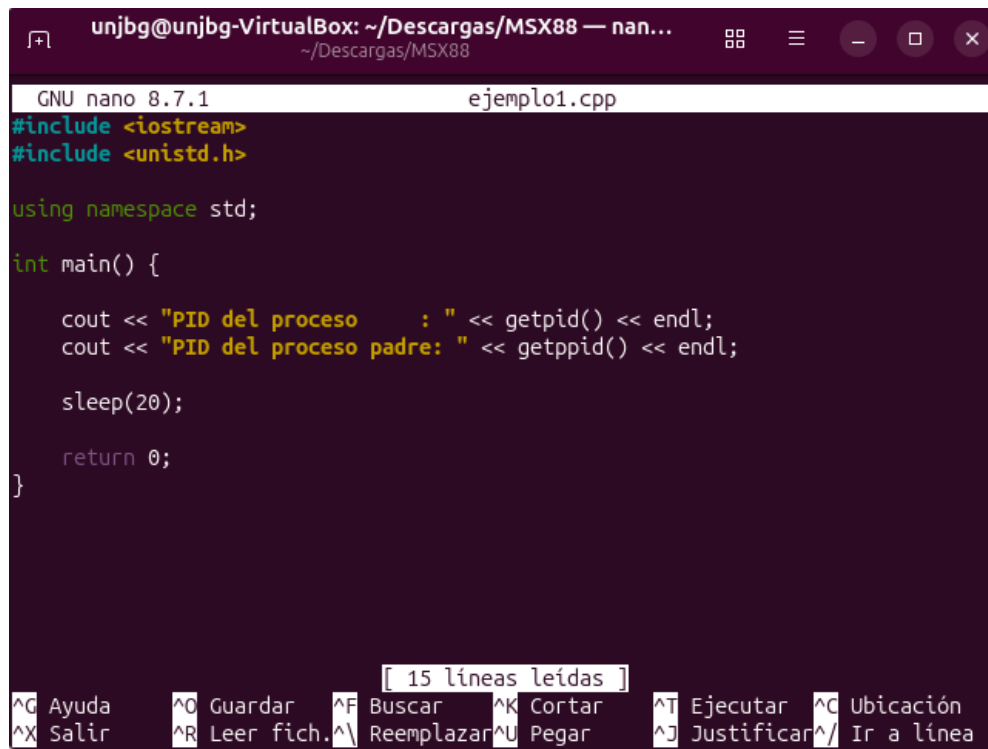
```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ g++ ejemplo1.cpp -o ejemplo1
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ ./ejemplo1
PID del proceso: 10179
```

Ejecutamos el comando ps -e desde otra terminal y localizamos el proceso mientras todavía se encuentra ejecutándose.

```
unjbg@unjbg-VirtualBox: ~
9445 ?      00:00:01 kworker/u17:1-writeback
9496 ?      00:00:00 kworker/0:1-events
9643 ?      00:00:00 bwrap
9644 ?      00:00:00 bwrap
9646 ?      00:00:00 glycin-image-rs
9677 ?      00:00:03 loupe
9694 ?      00:00:00 bwrap
9699 ?      00:00:00 bwrap
9700 ?      00:00:00 glycin-image-rs
9755 ?      00:00:00 bwrap
9756 ?      00:00:00 bwrap
9759 ?      00:00:00 glycin-image-rs
9839 ?      00:00:00 bwrap
9840 ?      00:00:00 bwrap
9842 ?      00:00:00 glycin-image-rs
9990 ?      00:00:00 kworker/2:3-cgroup_free
10002 ?     00:00:00 kworker/u18:1-events_unbound
10131 ?     00:00:00 bwrap
10132 ?     00:00:00 bwrap
10133 ?     00:00:00 glycin-svg
10156 pts/1   00:00:00 bash
10179 pts/0   00:00:00 ejemplo1
10182 pts/1   00:00:00 ps
unjbg@unjbg-VirtualBox:~$
```

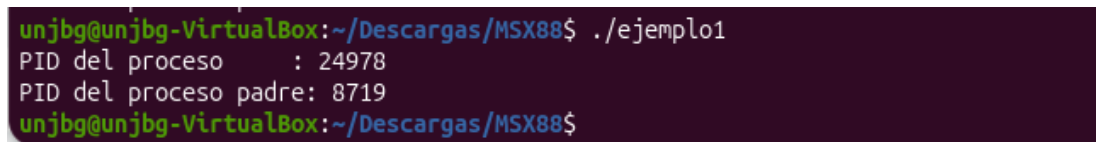
Parte 2: Procesos Padres e Hijos

Modificamos el programa del ejercicio1.cpp



```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nan...  
~ / Descargas/MSX88  
GNU nano 8.7.1 ejemplo1.cpp  
#include <iostream>  
#include <unistd.h>  
  
using namespace std;  
  
int main() {  
  
    cout << "PID del proceso      : " << getpid() << endl;  
    cout << "PID del proceso padre: " << getppid() << endl;  
  
    sleep(20);  
  
    return 0;  
}  
  
[ 15 líneas leídas ]  
^G Ayuda      ^O Guardar    ^F Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación  
^X Salir      ^R Leer fich. ^\ Reemplazar  ^U Pegar      ^J Justificar ^_ Ir a línea
```

Compilamos y ejecutamos nuevamente el programa.



```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ ./ejemplo1  
PID del proceso      : 24978  
PID del proceso padre: 8719  
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$
```

Ejecutamos el comando ps -ef desde otra terminal.

```
unjbg@unjbg-VirtualBox: ~  
unjbg 9842 9840 0 09:45 ? 00:00:00 /usr/libexec/glycin-loaders/  
root 10002 2 0 09:51 ? 00:00:02 [kworker/u18:1-events_unboun  
unjbg 10156 8683 0 09:55 pts/1 00:00:00 /usr/bin/bash  
root 10185 2 0 09:56 ? 00:00:00 [kworker/u17:3-events_power_  
unjbg 10594 9677 0 09:57 ? 00:00:00 bwrap --unshare-all --die-wi  
unjbg 10595 10594 0 09:57 ? 00:00:00 bwrap --unshare-all --die-wi  
unjbg 10597 10595 0 09:57 ? 00:00:00 /usr/libexec/glycin-loaders/  
root 11283 2 0 09:57 ? 00:00:01 [kworker/u20:2-events_power_  
root 12355 2 0 09:58 ? 00:00:00 [kworker/3:1-mm_percpu_wq]  
unjbg 13726 9677 0 09:58 ? 00:00:00 bwrap --unshare-all --die-wi  
unjbg 13729 13726 0 09:58 ? 00:00:00 bwrap --unshare-all --die-wi  
unjbg 13730 13729 0 09:58 ? 00:00:00 /usr/libexec/glycin-loaders/  
root 14735 2 0 10:00 ? 00:00:00 [kworker/2:0-events]  
root 23754 2 0 10:00 ? 00:00:00 [kworker/0:2-cgroup_release]  
root 24424 2 0 10:01 ? 00:00:00 [psimon]  
unjbg 24764 9677 0 10:03 ? 00:00:00 bwrap --unshare-all --die-wi  
unjbg 24765 24764 0 10:03 ? 00:00:00 bwrap --unshare-all --die-wi  
unjbg 24767 24765 0 10:03 ? 00:00:00 /usr/libexec/glycin-loaders/  
root 24958 2 0 10:06 ? 00:00:00 [kworker/3:0]  
root 24959 2 0 10:06 ? 00:00:00 [kworker/3:3]  
root 24960 2 0 10:06 ? 00:00:00 [kworker/0:1-cgroup_free]  
unjbg 24978 8719 0 10:07 pts/0 00:00:00 ./ejemplo1  
unjbg 24987 10156 0 10:07 pts/1 00:00:00 ps -ef  
unjbg@unjbg-VirtualBox:~$
```

Actividad 2

Determine:

1. El PID del programa

El valor del PID obtenido fue de 24978

2. El PPID del programa

El valor del PPID obtenido fue de 8719

3. El proceso que corresponde a PPDI obtenido

Corresponde al proceso bash, es decir al intérprete de comandos (shell)

4. La relación existente entre el programa ejecutado y el terminal o shell desde el cual fue iniciado

Es un proceso hijo del terminal que actúa como su proceso padre

Parte 3: Creación de un proceso con fork()

Modificamos el programa ejemplo2.cpp.

```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nan...
~/Descargas/MSX88
GNU nano 8.7.1 ejemplo2.cpp
#include <iostream>
#include <unistd.h>

using namespace std;

int main() {
    fork();

    cout << "Hola Linux. PID = " << getpid() << endl;

    return 0;
}

[ 13 líneas leídas ]
^G Ayuda      ^O Guardar    ^F Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich.^_ Reemplazar  ^U Pegar      ^J Justificar ^/ Ir a línea
```

Compilamos y ejecutamos el programa.

```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ g++ ejemplo2.cpp -o ejemplo2
./ejemplo2
Hola Linux. PID = 5357
Hola Linux. PID = 5358
```

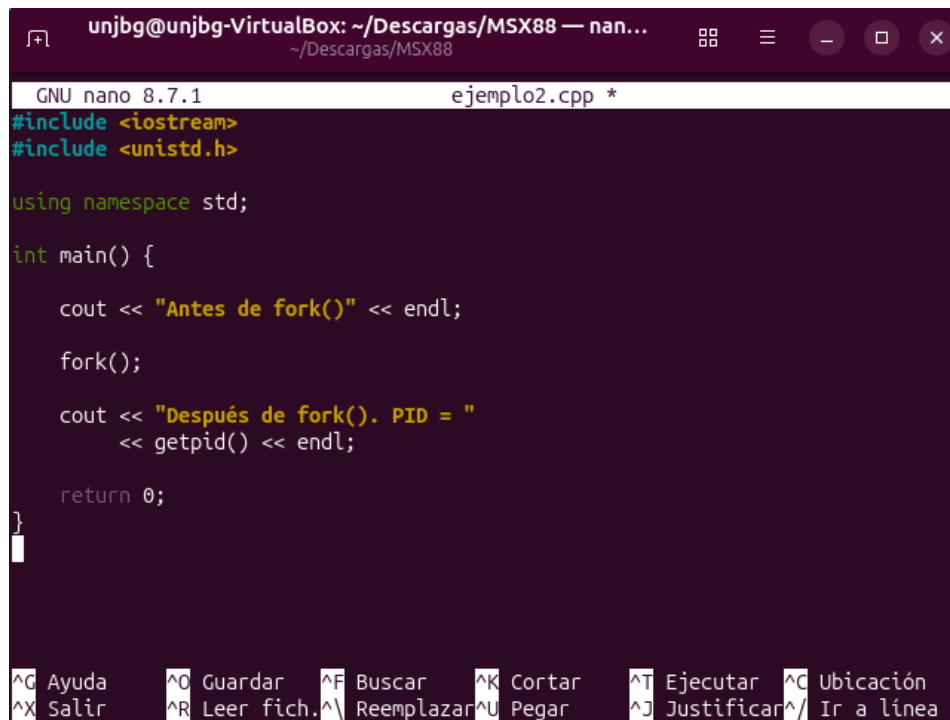
Actividad 3

- ¿Cuántas veces aparece “Hola Linux”? Explique la razón. Identifique además, los diferentes PID mostrados

Aparece 2 veces porque la función `fork()` crea un proceso hijo. Tanto el proceso padre como el proceso hijo ejecutan la instrucción `cout` que se encuentra después de `fork()`. Los PID mostrados son 5357 y 5358, correspondientes a los dos procesos generados.

Parte 4: Instrucciones antes y después de fork()

Modificamos el programa ejemplo2.cpp.



```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nan...
~/Descargas/MSX88
GNU nano 8.7.1 ejemplo2.cpp *
#include <iostream>
#include <unistd.h>

using namespace std;

int main() {

    cout << "Antes de fork()" << endl;

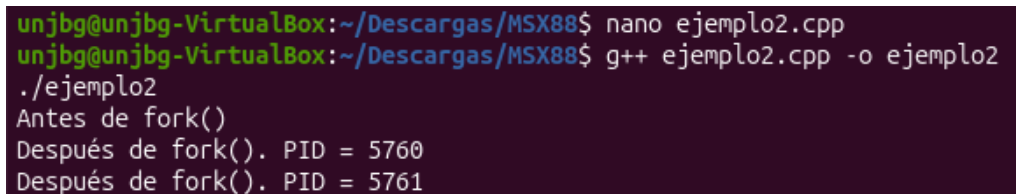
    fork();

    cout << "Después de fork(). PID = "
         << getpid() << endl;

    return 0;
}

^G Ayuda  ^O Guardar  ^F Buscar  ^K Cortar  ^T Ejecutar  ^C Ubicación
^X Salir  ^R Leer fich. ^_ Reemplazar ^U Pegar  ^J Justificar ^_ Ir a línea
```

Compilamos y ejecutamos el programa.



```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ nano ejemplo2.cpp
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ g++ ejemplo2.cpp -o ejemplo2
./ejemplo2
Antes de fork()
Después de fork(). PID = 5760
Después de fork(). PID = 5761
```

Actividad 4

Responda:

1. ¿Cuántas veces se muestra “Antes de fork()”?

Se muestra una vez, porque antes de ejecutar fork() existe un solo proceso.

2. ¿Cuántas veces se muestra “Después de fork()”?

Se muestra dos veces, porque fork() crea un proceso hijo y tanto el padre como el hijo continúan ejecutando las instrucciones posteriores.

3. ¿Por qué ocurre esta diferencia?

La diferencia ocurre porque fork() duplica el proceso. La instrucción anterior a fork() es ejecutada únicamente por el proceso original, mientras que las instrucciones posteriores son ejecutadas por el padre y el hijo.

4. ¿El orden en que aparecen los mensajes del padre y del hijo siempre es el mismo? Explique.

No. El orden puede variar, porque el sistema operativo decide cuál de los dos procesos se ejecuta primero. Por ello, el padre puede mostrar su mensaje antes que el hijo o viceversa.

Parte 5: Múltiples llamadas a fork()

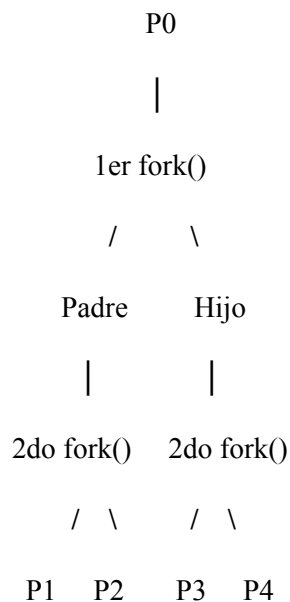
Actividad 5

Antes de ejecutar el programa:

1. Prediga cuantos procesos existirán después del segundo fork()

Después del segundo fork() existirán 4 procesos.

2. Dibuje el árbol de procesos que se generará



3. Prediga cuantas veces se imprimirá la última instrucción cout

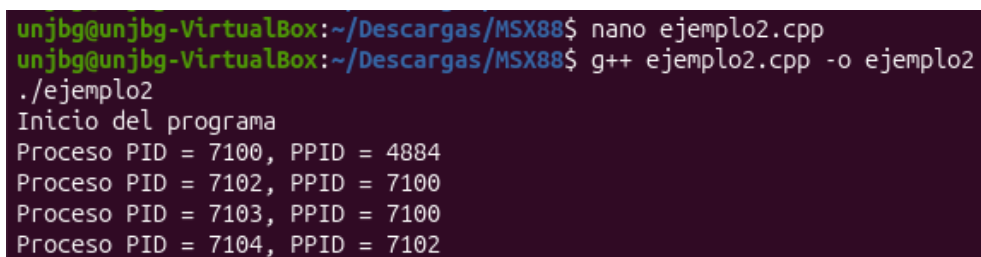
La última instrucción cout se imprimirá 4 veces, una vez por cada proceso existente después del segundo fork().

Modificamos el programa ejemplo2.cpp.



```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nan...  
~/Descargas/MSX88  
GNU nano 8.7.1 ejemplo2.cpp  
#include <iostream>  
#include <unistd.h>  
  
using namespace std;  
  
int main() {  
  
    cout << "Inicio del programa" << endl;  
  
    fork();  
    fork();  
  
    cout << "Proceso PID = "  
        << getpid()  
        << ", PPID = "  
        << getppid()  
        << endl;  
  
    return 0;  
}  
[ 21 líneas leídas ]  
^G Ayuda      ^O Guardar    ^F Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación  
^X Salir      ^R Leer fich. ^\ Reemplazar  ^U Pegar      ^J Justificar ^_ Ir a línea
```

Compilamos y ejecutamos el programa.



```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ nano ejemplo2.cpp  
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ g++ ejemplo2.cpp -o ejemplo2  
./ejemplo2  
Inicio del programa  
Proceso PID = 7100, PPID = 4884  
Proceso PID = 7102, PPID = 7100  
Proceso PID = 7103, PPID = 7100  
Proceso PID = 7104, PPID = 7102
```

Comparamos el resultado obtenido con su predicción.

Al ejecutar el programa se comprobó la predicción realizada, obteniéndose cuatro procesos después del segundo fork() y cuatro impresiones de la instrucción cout, cada una con un PID diferente.

Parte 6: Diferenciación entre padre e hijo

Modificamos el programa ejemplo3.cpp.



```
GNU nano 8.7.1 ejemplo3.cpp *
#include <iostream>
#include <unistd.h>

using namespace std;

int main() {

    pid_t pid = fork();

    if (pid < 0) {

        cerr << "Error al crear el proceso hijo" << endl;
        return 1;

    } else if (pid == 0) {

        cout << "HIJO"
              << " | PID = " << getpid()
              << " | PPID = " << getppid()
              << endl;

    } else {

        cout << "PADRE"
              << " | PID = " << getpid()
              << " | PID del hijo = " << pid
              << endl;

    }

    return 0;
}
```

Compilamos y ejecutamos el programa 3 veces.

```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ nano ejemplo3.cpp
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ g++ ejemplo3.cpp -o ejemplo3
./ejemplo3unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ ./ejemplo3
HIJO | PID = 4612 | PPID = 4611
PADRE | PID = 4611 | PID del hijo = 4612
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ ./ejemplo3
PADRE | PID = 4615 | PID del hijo = 4617
HIJO | PID = 4617 | PPID = 4615
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ ./ejemplo3
HIJO | PID = 4663 | PPID = 4662
PADRE | PID = 4662 | PID del hijo = 4663
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$
```

Actividad 6

Responda:

1. ¿Qué valor retorna fork() al proceso hijo?

Al proceso hijo, `fork()` le retorna el valor **0**, lo que permite identificar que se trata del proceso hijo.

2. ¿Qué valor retorna `fork()` al proceso padre?

Al proceso padre, `fork()` le retorna el PID del proceso hijo creado. Por ejemplo, en las ejecuciones realizadas se obtuvieron los PID de hijo 4612, 4617 y 4663.

3. ¿El PID mostrado por el hijo coincide con el valor retornado por `fork()` al padre?

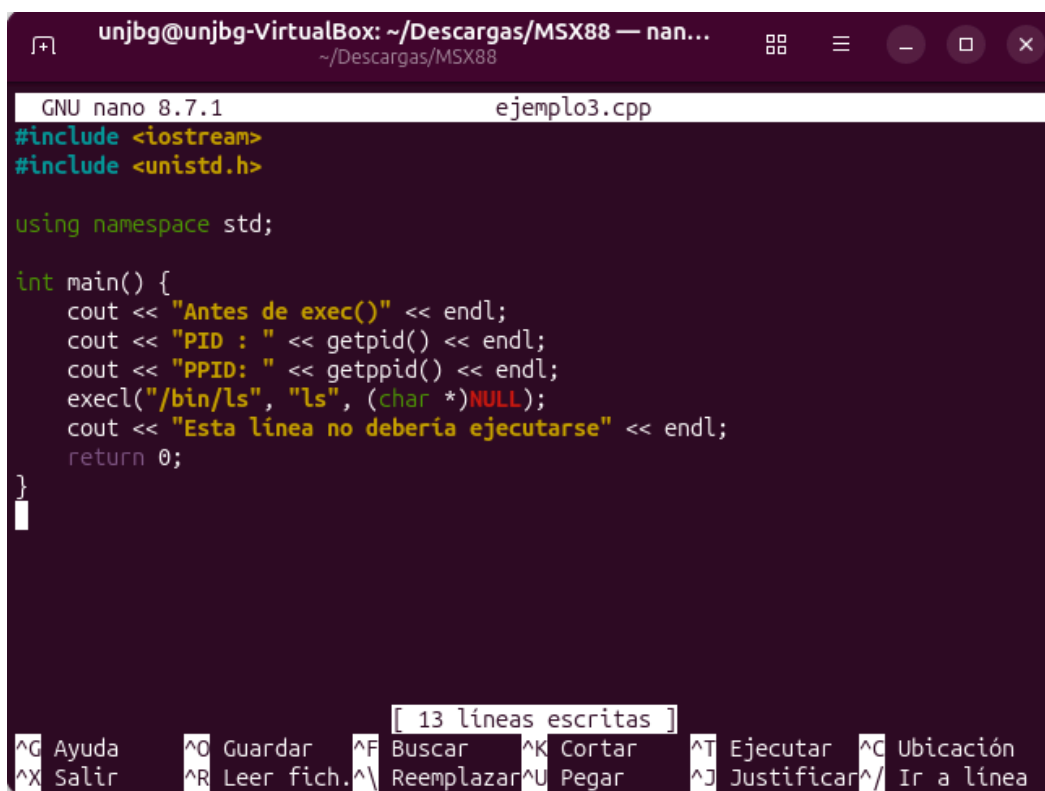
Sí. El valor retornado por `fork()` al proceso padre coincide con el PID del proceso hijo. Esto se observa, por ejemplo, cuando el padre muestra PID del hijo = 4612 y el hijo muestra PID = 4612.

4. ¿El mensaje del padre siempre aparece primero? Justifique.

No. El mensaje del padre no siempre aparece primero. En las ejecuciones realizadas, algunas veces apareció primero el mensaje del hijo y otras veces el del padre. Esto se debe a que el sistema operativo determina qué proceso se ejecuta primero según la planificación del procesador.

Parte 7: Reemplazo de un proceso proceso mediante `exec()`

Modificamos el programa `ejemplo3.cpp`.



```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nan...
GNU nano 8.7.1 ejemplo3.cpp
#include <iostream>
#include <unistd.h>

using namespace std;

int main() {
    cout << "Antes de exec()" << endl;
    cout << "PID : " << getpid() << endl;
    cout << "PPID: " << getppid() << endl;
    execl("/bin/ls", "ls", (char *)NULL);
    cout << "Esta línea no debería ejecutarse" << endl;
    return 0;
}
```

[13 líneas escritas]

^G Ayuda ^O Guardar ^F Buscar ^K Cortar ^T Ejecutar ^C Ubicación
^X Salir ^R Leer fich. ^\ Reemplazar ^U Pegar ^J Justificar ^_ Ir a línea

Compilamos y ejecutamos el programa.

```

unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ nano ejemplo3.cpp
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ g++ ejemplo3.cpp -o ejemplo3
./ejemplo3
Antes de exec()
PID : 6362
PPID: 5978
ASM88.EXE    DEMOMEMO.EJE  LINK88.EXE   ejemplo1     ejemplo3.cpp
DEMO.EXE     DEMOPIO.EJE   MSX88.EXE    ejemplo1.cpp  hola
DEMODMA.EJE  DEMOV40.EXE  README.TXT   ejemplo3      hola.c

```

Actividad 7

Explique:

1. ¿Qué programa se ejecuta después de `execl()`?

Se ejecuta el programa `ls`, que muestra el contenido de la carpeta

2. ¿Se crea un nuevo proceso mediante `exec()`?

No. `execl()` reemplaza el programa actual dentro del mismo proceso

3. ¿Por qué no se muestra el mensaje: Esta línea no debería ejecutarse?

Porque `exec()` reemplaza todo el programa original por `ls`, por lo que las instrucciones posteriores ya no se ejecutan

4. ¿Qué ocurre con el programa original cuando `execl()` se ejecuta correctamente?

El programa original es reemplazado por `ls`, manteniendo el mismo PID del proceso

Parte 8: Combinación de `fork()`, `exec()` y `wait()`

Modificamos el programa `ejemplo4.cpp`.

```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nan...
~/Descargas/MSX88
GNU nano 8.7.1 ejemplo4.cpp
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>

using namespace std;

int main() {
    pid_t pid = fork();
    if (pid < 0) {
        cerr << "No fue posible crear el proceso hijo" << endl;
        return 1;
    } else if (pid == 0) {
        cout << "Hijo: ejecutando ls..." << endl;
        execl("/bin/ls", "ls", (char *)NULL);
        cerr << "Error al ejecutar ls" << endl;
        return 1;
    } else {
        cout << "Padre: esperando al proceso hijo..." << endl;
        wait(NULL);
        cout << "Padre: el proceso hijo ha terminado."
              << endl;
    }
    return 0;
}
```

[24 líneas escritas]

^G Ayuda ^O Guardar ^F Buscar ^K Cortar ^T Ejecutar ^C Ubicación
^X Salir ^R Leer fich. ^A Reemplazar ^U Pegar ^J Justificar ^_ Ir a línea

Compilamos y ejecutamos el programa.

```
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ nano ejemplo4.cpp
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ g++ ejemplo4.cpp -o ejemplo4
./ejemplo4
Padre: esperando al proceso hijo...
Hijo: ejecutando ls...
ASM88.EXE    DEMOMEMO.EJE  LINK88.EXE  ejemplo1    ejemplo3.cpp  hola
DEMO.EXE     DEMOPIO.EJE  MSX88.EXE  ejemplo1.cpp ejemplo4      hola.c
DEMOMA.EJE   DEMOV40.EXE  README.TXT ejemplo3     ejemplo4.cpp
Padre: el proceso hijo ha terminado.
```

Actividad 8

Analice el programa e identifique:

1. ¿En qué momento se crea el hijo?

El proceso hijo se crea cuando se ejecuta la instrucción `fork()`

2. ¿Qué proceso ejecuta `ls`?

El proceso hijo ejecuta `ls` mediante la función `execl()`

3. ¿Qué proceso ejecuta `wait()`?

El proceso padre ejecuta `wait()` para esperar al proceso hijo

4. ¿Qué sucede con el padre mientras el hijo ejecuta ls?

El padre queda esperando hasta que el proceso hijo termine

5. ¿Por qué el mensaje: Padre: el proceso hijo ha terminado. se muestra después de finalizar ls?

Porque wait() hace que el padre espere hasta que el proceso hijo termine

IV. ACTIVIDAD DE APLICACIÓN

Desarrolle un programa en C/C++ que implemente un intérprete básico de comandos. El programa deberá:

1. Mostrar un prompt: SO>
2. Leer la entrada estándar, el nombre de un programa. Por ejemplo: SO> ls
3. Crear un proceso hijo mediante: fork()
4. El proceso hijo deberá ejecutar el comando indicado utilizando una función de la familia: exec()
5. Los programas deberán buscarse inicialmente en el directorio: /bin. Puede utilizarse comandos que no requieren argumentos, tales como: ls, date, pwd, uname, who
6. El proceso padre deberá utilizar wait() o waitpid() para esperar a que termine el hijo. Por tanto, no deberá solicitarse un nuevo comando hasta que el anterior haya terminado.
7. El intérprete continuará solicitando comandos hasta que el usuario introduzca: salir.
8. El ejemplo deberá controlar, como mínimo:
 - errores producidos por fork();
 - errores producidos por exec();
 - finalización correcta del proceso hijo

Creamos el programa interprete.cpp

```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88 — nan...
~/Descargas/MSX88
GNU nano 8.7.1 interprete.cpp
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

using namespace std;

int main() {
    string comando;

    while (true) {
        cout << "SO> ";
        cin >> comando;

        if (comando == "salir") {
            cout << "Finalizando interprete..." << endl;
            break;
        }

        pid_t pid = fork();

        if (pid < 0) {
            cerr << "Error al crear el proceso hijo." << endl;
            continue;
        }

        if (pid == 0) {
            // Proceso hijo
            string ruta = "/bin/" + comando;

            execl(ruta.c_str(), comando.c_str(), (char *)NULL);

            // Solo se ejecuta si exec falla
            cerr << "Error al ejecutar el comando: " << comando << endl;
            return 1;
        } else {
            // Proceso padre
            int estado;
            waitpid(pid, &estado, 0);
        }
    }

    return 0;
}

^G Ayuda      ^O Guardar    ^F Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich. ^\ Reemplazar ^U Pegar      ^J Justificar ^_ Ir a línea
```

Compilamos y ejecutamos los comandos: date, pwd, ls, uname, who y por último salir consecutivamente.

```
unjbg@unjbg-VirtualBox: ~/Descargas/MSX88
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ nano interprete.cpp
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ g++ interprete.cpp -o interprete
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$ ./interprete
S0> date
vie 18 sept 2026 15:18:23 -05
S0> pwd
/home/unjbg/Descargas/MSX88
S0> ls
ASM88.EXE      DEMOPIO.EJE  EJ1.0        ejemplo2      ejemplo3.cpp
DEMO.EXE       DEMOV40.EXE LINK88.EXE    ejemplo2.cpp  interprete
DEMODMA.EJE    EJ1.ASM     MSX88.EXE    ejemplo3      interprete.cpp
DEMOMEMO.EJE  EJ1.EJE     README.TXT   ejemplo3.com
S0> uname
Linux
S0> who
S0> salir
Finalizando interprete...
unjbg@unjbg-VirtualBox:~/Descargas/MSX88$
```

V. ANÁLISIS DE RESULTADOS

Responda fundamentalmente las siguientes preguntas:

1. ¿Cuál es la diferencia entre el PID y el PPID de un proceso?

El PID identifica de forma única a un proceso, mientras que el PPID identifica al proceso padre que lo creó

2. Explique que sucede con un programa cuando se ejecuta: fork(); ¿Qué información permite diferenciar al proceso padre del proceso hijo?

El fork() crea un nuevo proceso hijo. Para diferenciar al padre del hijo en el proceso hijo retorna a 0, mientras que en el proceso padre retorna el PID del proceso hijo.

3. Explique la diferencia fundamental entre fork() y exec(). ¿Ambas funciones crean un nuevo proceso?

fork() crea un nuevo proceso a partir del proceso existente. exec() no crea un proceso nuevo sino que reemplaza el programa actual por otro.

4. ¿Por qué es necesario utilizar wait() en el intérprete de comandos de desarrollo? ¿Qué comportamiento tendría el programa si se elimina esta llamada?

wait() permite que el proceso padre espere a que termine el proceso hijo. Si se llega a eliminar el proceso padre continuará ejecutándose antes de que termine el proceso hijo lo que provocaría un desorden en la ejecución.

VI. CONCLUSIONES

- Se logró identificar los procesos en el sistema Linux mediante sus identificadores PID y PPID, comprobando que cada proceso posee un identificador único y que el PPID permite reconocer el proceso padre.

- Se comprendió la relación entre un proceso padre y su proceso hijo, verificando que el PPID del hijo corresponde al PID del proceso padre que lo creó.
- Se logró crear nuevos procesos mediante la llamada al sistema `fork()`, comprobando que una única ejecución del programa puede generar procesos adicionales.
- Se interpretó correctamente el valor retornado por `fork()`, identificando que el proceso hijo recibe el valor 0, mientras que el proceso padre recibe el PID del proceso hijo, permitiendo diferenciar ambas ejecuciones.
- Se comprendió el funcionamiento de la familia de funciones `exec()`, reconociendo que permite reemplazar la imagen del proceso actual por otro programa, manteniendo el mismo proceso mientras cambia el programa que ejecuta.
- Se comprendió la importancia de `wait()` y `waitpid()` para sincronizar los procesos, permitiendo que el proceso padre espere la finalización de su proceso hijo y evitando una ejecución desordenada o la permanencia innecesaria de procesos terminados.
- Se logró analizar el ciclo básico de los procesos en Linux mediante pequeños programas, observando su creación, ejecución, relación padre-hijo y finalización, así como la influencia de la planificación del sistema operativo en el orden de ejecución.